Candidates 2026
Propuesta de programación

Borrador para la junta del 19 de septiembre · Basado en el reglamento del 28 de agosto de 2026 · Todavía no es código: son ideas para discutir

Tres ideas antes de los detalles

- **El robot piensa en casillas.** Todo en las dos pistas mide 30 × 30 cm. Si el robot sabe avanzar exactamente una casilla y girar exactamente 90°, el problema deja de ser de física y se vuelve de estructuras de datos: una matriz para el mapa, una pila para regresar y una lista para el recorrido.
- **Un solo explorador para dos retos.** La búsqueda en profundidad (DFS) con pila resuelve el laberinto de la pista A y también la sección 1 de la pista B. Se programa una vez y se prueba en dos lugares.
- **Primero lo que suma en las dos pistas.** Moverse exacto y leer colores da puntos en A y en B. La cámara del ArUco va al final, porque pide hardware extra y no le quita puntos a nada si no sale.

Regla que el diseño respeta en todo: nada del mapa se guarda antes de la ronda, porque el reglamento prohíbe el pre-mapeo. El robot arranca cada ronda con la memoria vacía y aprende paredes y colores mientras avanza. Lo único que se guarda antes es la calibración de los sensores.

SECCIÓN 4 DEL REGLAMENTO

Dónde están los puntos

Pista A · Laberinto

ACCIÓN	PUNTOS
Detectar y mostrar colores (4 × 5)	20
Speedbumps (4 × 5)	20
Escaleras (3 × 15)	45
Rampa	20
Llegar al checkpoint final	25
Bonus 1: regresar por el mismo camino	35
Bonus 2: marcador ArUco	30
Máximo	**195**

Pista B · Niveles

ACCIÓN	SIN PELOTA	CON PELOTA
Sacar la pelota de su casilla	—	15
Checkpoint de la sección 1	40	70
Checkpoint de la sección 2	40	70
Final de la sección 3	50	85
Máximo	**130**	**240**

Lo que dicen estos números. En la pista A, 85 de los 130 puntos base son obstáculos (topes, escaleras, rampa): eso lo gana sobre todo la mecánica, no el código. En la pista B, la garra vale 110 de 240 puntos. Y cada ronda dura 6 minutos por pista, con el reloj corriendo aunque haya lack of progress.

LO QUE USAN LAS DOS PISTAS

La base: casillas y rumbos

El robot guarda dónde está como una casilla `(x, y)` y hacia dónde ve como un número del 0 al 3. Girar es sumar o restar 1; avanzar es sumar un vector. Con eso cada lectura de sensor se puede anotar en el mapa.



Cada casilla de la cuadrícula es una unidad de 30 × 30 cm (± 2). Los rumbos son de la **pista**, no del robot: "arriba" siempre es la misma dirección aunque el robot esté volteado.

El sensor de la derecha mira hacia `(rumbo + 1) mod 4` y el de la izquierda hacia `(rumbo + 3) mod 4`. Así una lectura "pared a la derecha" se convierte en "pared al este de esta casilla".

```
// Rumbo: 0 = arriba, 1 = derecha, 2 = abajo, 3 = izquierda
dx ← [0, 1, 0, -1]
dy ← [1, 0, -1, 0]

girarDerecha():      girar(+90°);  rumbo ← (rumbo + 1) mod 4
girarIzquierda():    girar(-90°);  rumbo ← (rumbo + 3) mod 4
avanzarUnaCasilla(): avanzar 30 cm con encoders, recto con el IMU
                     x ← x + dx[rumbo];  y ← y + dy[rumbo]

anotarParedes(x, y, rumbo):
    pared[x][y][rumbo]             ← distanciaFrente    < 20 cm
    pared[x][y][(rumbo + 1) mod 4] ← distanciaDerecha   < 20 cm
    pared[x][y][(rumbo + 3) mod 4] ← distanciaIzquierda < 20 cm

// Color: se calibra en los 2 minutos de cada ronda; los tonos cambian
calibrar():  poner el sensor sobre cada color y guardar su (R, G, B)
leerColor(): tomar 5 lecturas; cada una se asigna a la referencia más cercana;
             gana el color que más se repite
```

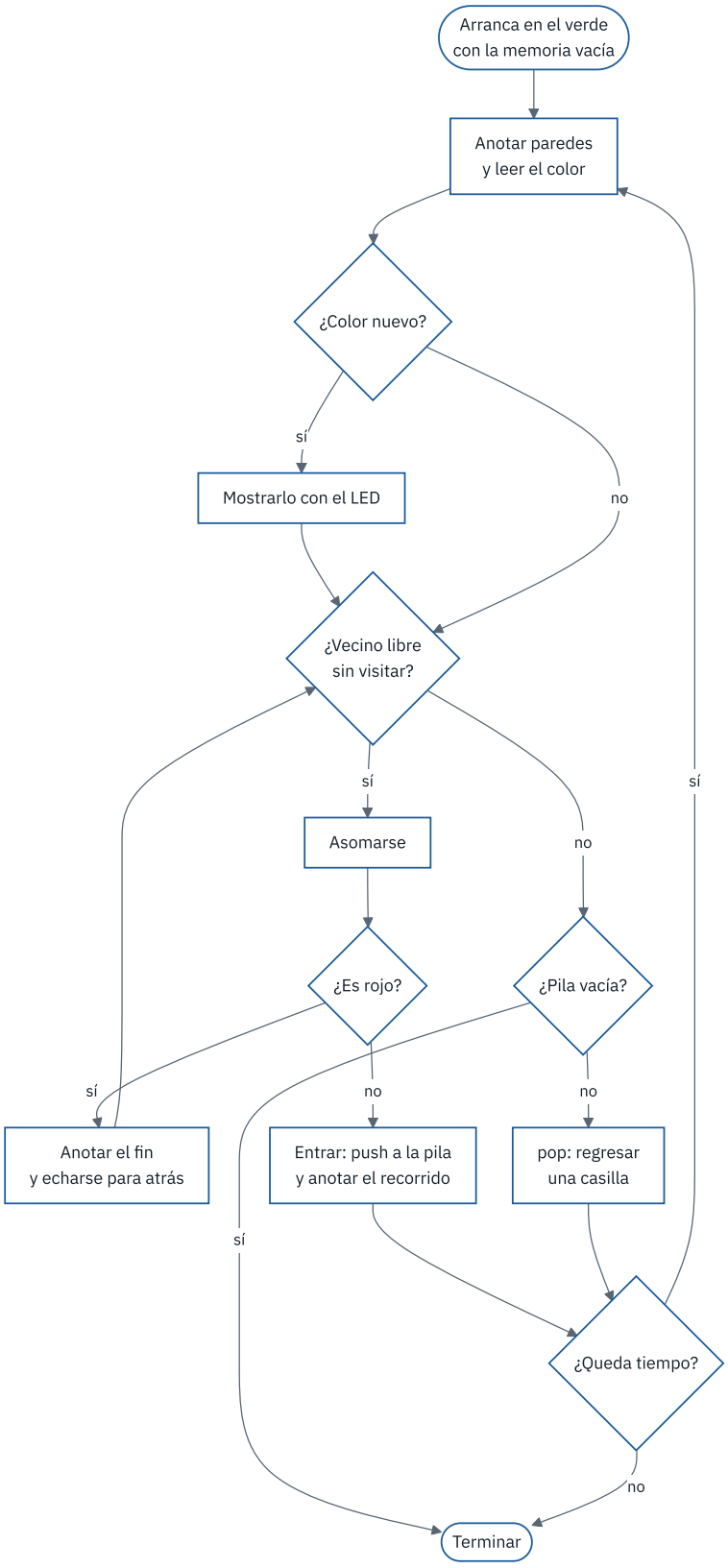
Por qué calibrar y no fijar umbrales. El reglamento dice que los tonos pueden cambiar y que entre casillas de distinto color hay una franja blanca. Comparar contra referencias tomadas ese mismo día, y votar entre 5 lecturas, aguanta las dos cosas. Los números (20 cm, 5 lecturas) son de ejemplo: se ajustan con pruebas.

SECCIÓN 3.1 DEL REGLAMENTO

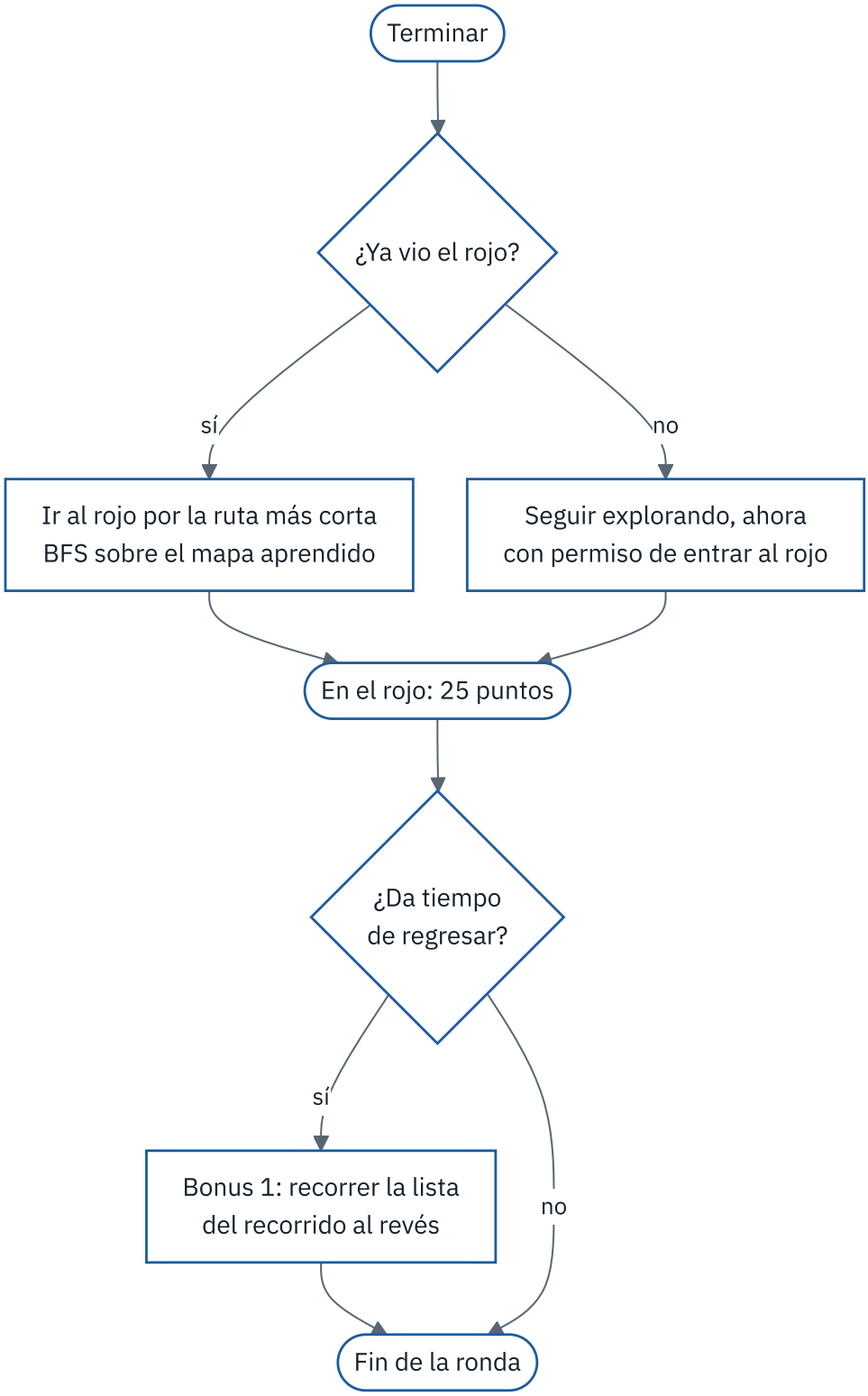
Pista A · Laberinto 5 × 5

El robot sale del verde, tiene que pasar por la mayor cantidad de casillas de color y llegar al rojo. El punto delicado: **al llegar al rojo se acaba la ronda**. Si el robot lo pisa a la mitad de la exploración, pierde los colores que le faltaban. Por eso explora todo primero, y el rojo lo deja para el final.

Cian Amarillo Naranja Rosa 5 puntos cada uno, una vez Inicio Fin



Parte 1, explorar. "Asomarse" es avanzar unos centímetros, leer el color de la casilla de enfrente y decidir si se entra.



Parte 2, terminar. Se llega aquí cuando ya no hay nada que explorar o cuando se acaba el tiempo reservado para explorar.

```
// Memoria del robot. Empieza vacía en cada ronda.
visitada[9][9]  ← falso        // 9×9: no sabemos en qué orilla arranca
pared[9][9][4]  ← desconocida
pila            ← vacía        // el camino para regresar hacia el inicio
recorrido       ← [(4, 4)]     // cada casilla pisada, en orden: es para el bonus
1
coloresVistos   ← { }
fin             ← ninguno      // dónde está la casilla roja, cuando la vea

x, y, rumbo ← 4, 4, ARRIBA
visitada[x][y] ← verdadero

mientras quedaTiempoParaExplorar():        // p. ej. 3:30 de 6:00; se ajusta
    anotarParedes(x, y, rumbo)
    c ← leerColor()
    si c es CIAN, AMARILLO, NARANJA o ROSA y c no está en coloresVistos:
        mostrarColor(c);  coloresVistos.agregar(c)

    d ← vecinoAbierto(x, y)     // sin pared, sin visitar y que no sea el fin
    si d existe:
        girarHacia(d)
        si asomarse() = ROJO:
            fin ← vecino(x, y, d);  echarseParaAtras()
        si no:
            terminarDeEntrar()
            pila.push(d);  (x, y) ← vecino(x, y, d)
            visitada[x][y] ← verdadero;  recorrido.agregar((x, y))
    si no, si la pila no está vacía:
        d ← pila.pop()                       // callejón sin salida: regresar por
donde vino
        girarHacia(opuesta(d));  avanzarUnaCasilla()
        recorrido.agregar((x, y))
    si no:
        salir del ciclo                       // pila vacía: ya exploró todo lo
que se puede

// Llegar al final
si fin = ninguno:  seguir explorando, ahora con permiso de entrar al rojo
si no:             ir por bfs(x, y, fin)      // ruta más corta; también va al
recorrido

// Bonus 1: pasar por las mismas casillas en orden inverso
```

```
si tiempoRestante() > largo(recorrido) × SEG_POR_CASILLA:
    para cada casilla en recorrido, del final al principio:  moverseA(casilla)
```

Por qué DFS con pila. Explora todo el laberinto sin perderse: la pila siempre guarda cómo regresar, así que cuando el robot llega a un callejón sin salida solo hace pop y vuelve un paso. Cuando la pila se vacía, ya vio todo lo que se puede ver. Es exactamente la estructura que vemos en la materia de estructuras de datos.

Por qué asomarse antes de entrar. Una casilla cuenta como alcanzada cuando el robot tiene al menos la mitad adentro. Si el sensor de color va en la orilla de enfrente, lee la casilla siguiente cuando el robot todavía no ha metido ni la mitad, y puede echarse para atrás si es la roja.

Por qué guardar el recorrido completo. El bonus 1 (35 puntos) pide regresar por las mismas casillas en orden inverso. Si cada casilla pisada se anota en una lista, el regreso es leer la lista al revés. Lo que no está claro es si cuentan las casillas de los callejones: con ellas el regreso puede ser de unas 50 casillas; sin ellas, de unas 10 (ver la pregunta 1 al final). Por eso el robot decide al llegar si le alcanza el tiempo.

Rampa, escaleras y topes. Los topes miden 6 mm y los escalones 5 mm: los sube la mecánica, y el software solo corrige el rumbo con el IMU si el robot se desvía. La rampa sí pide lógica: mide 60 cm, o sea 2 casillas. Cuando el IMU marca inclinación, el robot sigue recto sin anotar paredes y, al volver a quedar plano, cuenta 2 casillas.

ArUco al final. Pide una cámara y un procesador que corra visión (el diccionario 4x4_50 ya viene en OpenCV), y mostrar el número al menos 3 segundos. Si falla, no quita puntos de nada más, así que es lo último que se construye.

Un dato para mecánica. Para girar dentro de una casilla sin tocar paredes, la diagonal del robot tiene que ser menor a 30 cm. Un robot de 20 × 20 cm ya tiene 28 cm de diagonal: muy justo.

SECCIÓN 3.2 DEL REGLAMENTO

Pista B · Niveles

Tres secciones seguidas, cada una con un truco distinto, y la pelota de golf (42 mm) en la garra todo el camino. El programa es una máquina de estados: una variable dice en qué sección va el robot, y entrar a una casilla roja lo pasa a la siguiente.

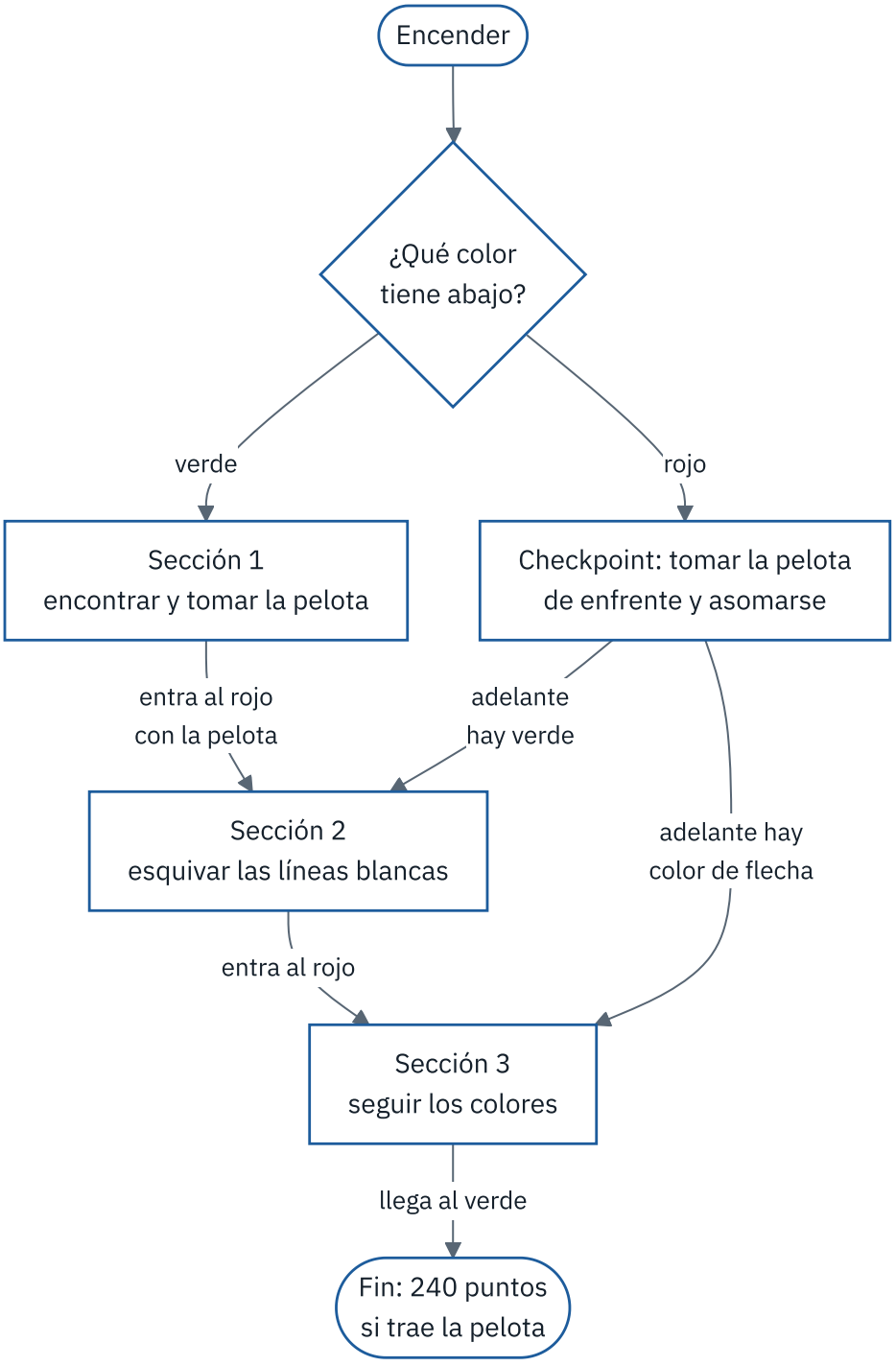


Diagrama de estados de la pista B. El arranque sirve igual al inicio de la ronda y después de cada lack of progress.

Al encender: ¿en qué sección estoy?

```
c ← leerColor()
si c = VERDE:  seccion ← 1                  // casilla de inicio
si c = ROJO:                                // checkpoint: viene de un lack of
progress
    tomarPelotaDeEnfrente()                // la ponen enfrente (imagen 3.1)
    adelante ← asomarse()
    si adelante = VERDE:  seccion ← 2
    si no:                seccion ← 3          // cian, amarillo, naranja o rosa
```

Por qué deducirlo del piso. Después de un lack of progress el capitán apaga y reinicia el robot, y la memoria se borra. En vez de guardar en qué sección iba, el robot lo deduce de lo que tiene abajo y enfrente. Así no dependemos de guardar nada entre reinicios, que podría confundirse con pre-mapeo.

Sección 1 · La pelota escondida

La pelota está en la casilla central de un cuadro de 3 × 3, rodeada por tres paredes; el lado abierto cambia cada ronda (4 posibilidades). La propuesta es no programar nada especial: usar el mismo DFS del laberinto, con otro objetivo.

```
tengoPelota ← falso
explorar con el DFS de la pista A mientras no tengoPelota:
    al asomarse a un vecino:
        si distanciaBaja < 25 cm y distanciaAlta > 30 cm:   // algo bajito: la
pelota
            abrirGarra()
            avanzar despacio hasta que sensorGarra detecte la pelota
            cerrarGarra();  tengoPelota ← verdadero
            retroceder una casilla              // la casilla de la pelota es un
callejón
        si ve ROJO:  anotarlo, pero no entrar todavía
ir al rojo  // BFS si ya lo vio; si no, seguir explorando
```

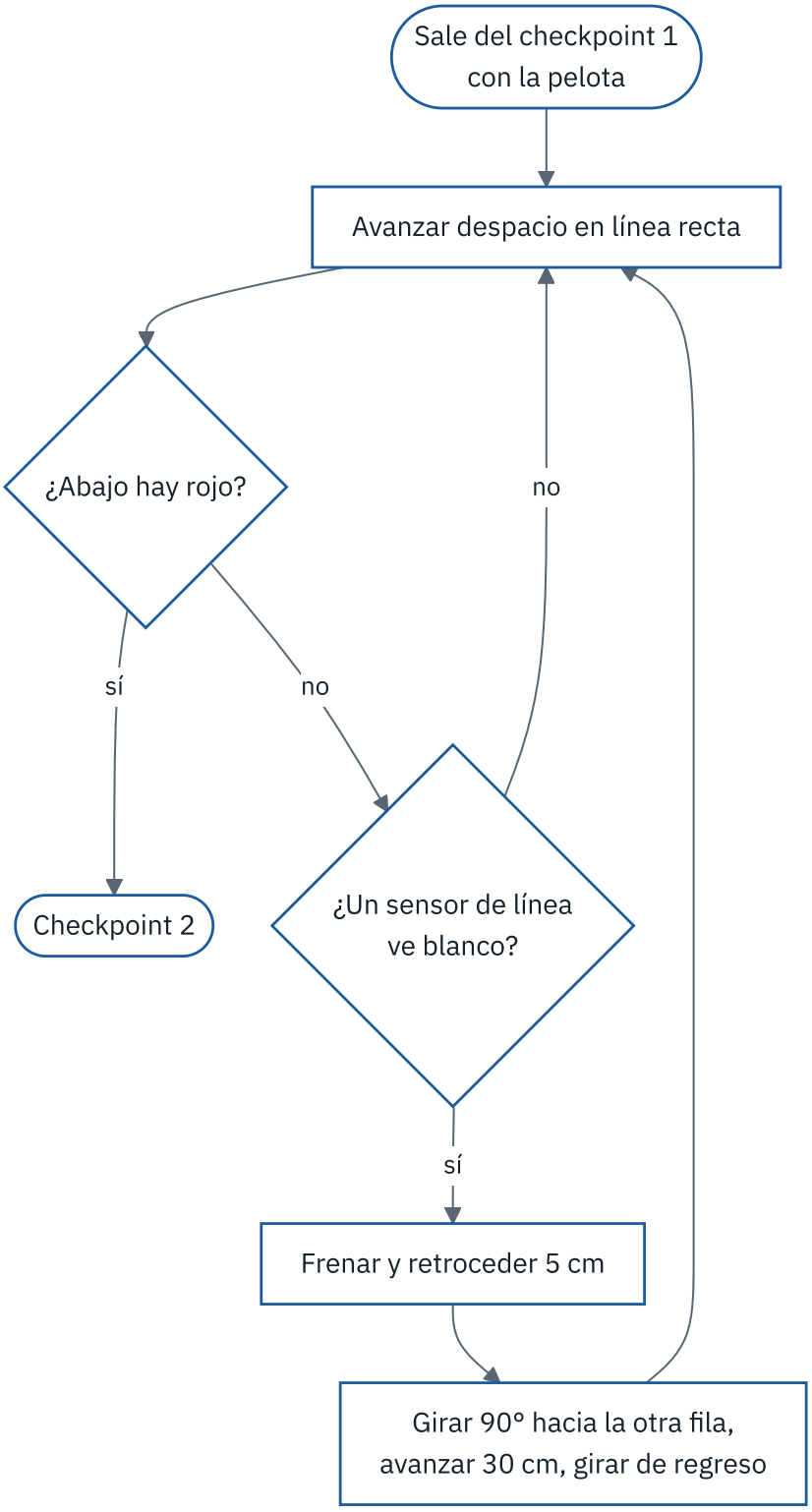
Por qué dos sensores de distancia a distinta altura. La pelota mide 4.2 cm y las paredes 15 cm. Un sensor a 2 cm del piso ve las dos cosas; uno a 8 cm solo ve la pared. Si el bajo ve algo cerca y el alto ve lejos, es la pelota. Y si el lado está cerrado, los dos ven pared: el DFS

lo trata como cualquier pared y busca otro lado. Las cuatro posiciones de las paredes quedan resueltas sin un solo caso especial.

Por qué no entrar al rojo antes. Llegar al checkpoint con pelota vale 70 y sin pelota 40. Es la misma regla del laberinto: el rojo se anota y se deja para cuando ya se cumplió el objetivo.

Sección 2 · Las líneas blancas

Una cuadrícula verde de 2 × 4 con líneas blancas de 2 cm pintadas en el piso. Cada línea tapa una de las dos filas y deja la otra libre (un hueco de 30 cm). Si más de media llanta pisa una línea, es lack of progress.


Sale del checkpoint 1 con la pelota
Avanzar despacio en línea recta
¿Abajo hay rojo?
sí
no
Checkpoint 2
no
¿Un sensor de línea ve blanco?
sí
Frenar y retroceder 5 cm
Girar 90° hacia la otra fila, avanzar 30 cm, girar de regreso

```
mientras leerColor() ≠ ROJO:
    avanzar despacio, recto con el IMU
    si lineaIzquierda ve BLANCO o lineaDerecha ve BLANCO:
        frenar();  retroceder(5 cm)
        girar 90° hacia la otra fila;  avanzar(30 cm);  girar 90° de regreso
        fila ← 1 - fila                    // solo hay dos filas
```

Por qué sensores de línea y no de distancia. Las líneas están pintadas, no son paredes: un sensor de distancia no las ve. Dos sensores infrarrojos mirando al piso, **delante de las llantas**, frenan al robot antes de que media llanta pise la línea.

Por qué funciona cambiar de fila. Si cada línea tapa solo una fila, en esa misma altura la otra fila siempre está libre. El robot no necesita saber dónde están las líneas: reacciona cuando se topa con una. Esto depende de que el acomodo sea siempre así (pregunta 3).

Sección 3 · Seguir los colores

Cada casilla de color dice a qué casilla pasar, y las direcciones son de la pista vista con el checkpoint abajo, **no del robot**.

Cian → derecha Amarillo → izquierda Naranja → arriba Rosa → abajo

Verde → fin

```
DIRECCION ← { CIAN: 1, AMARILLO: 3, NARANJA: 0, ROSA: 2 }   // mismos rumbos de la base
rumbo ← 0                   // al salir del checkpoint ve hacia "arriba"
avanzarUnaCasilla()
repetir:
    c ← leerColor()         // 5 lecturas en el centro de la casilla, gana la mayoría
    si c = VERDE:  detener();  salir        // llegó al fin
    nuevo ← DIRECCION[c]
    giro ← (nuevo - rumbo + 4) mod 4   // 0 recto · 1 derecha · 2 media vuelta · 3 izquierda
    girar(giro);  rumbo ← nuevo
    avanzarUnaCasilla()
```

Prueba de escritorio con el ejemplo del reglamento (imagen 2.7), cuya ruta es arriba, arriba, derecha, abajo, derecha:

PASO	COLOR LEÍDO	RUMBO ANTES	RUMBO NUEVO	GIRO QUE HACE EL ROBOT
1	Naranja	0 arriba	0 arriba	(0 − 0) = 0 → recto
2	Naranja	0 arriba	0 arriba	0 → recto
3	Cian	0 arriba	1 derecha	(1 − 0) = 1 → derecha
4	Rosa	1 derecha	2 abajo	(2 − 1) = 1 → derecha
5	Cian	2 abajo	1 derecha	(1 − 2 + 4) mod 4 = 3 → izquierda
6	Verde	Llegó al fin: se detiene		

Lo que enseña la tabla. En el paso 5 la casilla dice "derecha", pero el robot gira a la **izquierda**, porque viene bajando. Es justo la advertencia del reglamento. Guardar el rumbo como número hace que la resta diga siempre el giro correcto, sin una lista de casos.

PARA ELECTRÓNICA Y MECÁNICA

Lo que el software le pide al robot

PIEZA	PARA QUÉ LA USA EL PROGRAMA	DÓNDE VA
2 motores con encoder	Avanzar exactamente 30 cm	—
IMU (giroscopio)	Giros de 90°, ir recto, notar la rampa	Cerca del centro
3 sensores de distancia	Paredes al frente, izquierda y derecha	A más de 5 cm del piso, para que la rampa (unos 4 cm de alto) no parezca pared
1 sensor de distancia bajo	Ver la pelota	Al frente, a unos 2 cm del piso
Sensor de color	Leer la casilla y asomarse a la siguiente	Abajo, en la orilla delantera
2 sensores de línea (IR)	Líneas blancas de la sección 2	Abajo, delante de las llantas
Garra con servo y sensor	Tomar la pelota y saber que está adentro	Al frente
LED RGB y pantalla chica	Mostrar colores y el número del ArUco (3 s o más); también sirve para depurar	Visible para el juez
Botón y selector	Calibrar colores y elegir pista A o B	—
Cámara (al final)	Marcador ArUco	Mirando hacia las paredes

PROPUESTA DE ORDEN

En qué orden construirlo

Cada paso usa lo del anterior. Al terminar el paso 4 ya sumamos puntos en las dos pistas.

1. **Moverse exacto.** Una casilla recta y un giro de 90°. Prueba: recorrer un cuadrado de 4 casillas y regresar al mismo punto.
2. **Leer paredes y colores**, con la calibración de los 2 minutos.

3. **Laberinto con DFS:** ver colores y llegar al fin.
4. **Sección 3 de B:** seguir colores. Casi todo ya está hecho en los pasos 1 y 2.
5. **Sección 2 de B:** líneas blancas.
6. **Garra y sección 1 de B**, reusando el DFS.
7. **Bonus 1 del laberinto:** recorrer la lista al revés.
8. **ArUco.**

PARA HANS VELARDE O LUIS PACHUCA

Preguntas para los organizadores

1. **Bonus 1 de la pista A:** ¿"el mismo camino" incluye las casillas por donde el robot regresó al explorar callejones, o basta con la ruta de inicio a fin? Con la primera, el regreso puede ser de unas 50 casillas; con la segunda, de unas 10.
2. **Pista A:** si el robot pisa la casilla roja antes de terminar de explorar, ¿se acaba la ronda en ese momento?
3. **Sección 2:** ¿cada línea blanca tapa siempre una fila completa y deja la otra libre? ¿Hay paredes alrededor de la sección?
4. ¿Podemos usar librerías: drivers de sensores, y OpenCV para detectar el ArUco? El reglamento prohíbe código ajeno "para resolver alguna lógica y los movimientos".
5. ¿Cuánto tiempo tiene que verse el color en el LED para que cuente?
6. ¿De qué color es el piso normal del laberinto y del cuadro de la sección 1?

Borrador de trabajo del equipo. Los diagramas de flujo están escritos en Mermaid dentro de este archivo: se cambian editando el texto, sin programa de dibujo.